

Multi-Instance Paxos: Design Document

Based on Paxos Made Moderately Complex (PMMC) and Paxos Made Simple

1. Failure Model and Assumptions

Node Failures

- Servers fail only by **permanent crash** or **temporary network partition**. There is no Byzantine behavior — nodes do not lie, corrupt messages, or act maliciously.
- At most f servers may be unavailable at any time. With $2f + 1$ total servers, a majority quorum of $f + 1$ is always available among non-crashed nodes.
- There is **no server recovery**. A crashed node does not rejoin. Therefore, the system must be sized so that f permanent crashes still leave a majority.
- Clients are assumed to not crash while a request is in flight (or they retry).

Client Failures

- Clients broadcast requests to all servers. If a client crashes before receiving a reply, the request may or may not have been committed. This is acceptable because the deduplication mechanism (see §3) prevents double-execution.

Network Assumptions

- The network is **asynchronous**: messages may be arbitrarily delayed, reordered, or dropped, but are not corrupted.
- Liveness requires that eventually a **majority of servers can communicate** with each other and with the client. Safety holds unconditionally — no two servers will ever decide different values for the same log slot, regardless of network conditions.
- No authenticated channels are required; we assume an honest closed system.

Storage

- State is **in-memory only**. Neither the key-value store nor Paxos metadata are persisted to disk. A crash is permanent.

Time / Clocks

- No synchronized clocks are assumed. Timeouts are used only for **liveness** (triggering leader election and heartbeats), not for safety.

2. Protocol Sketch

2.1 Roles (Collocated in PaxosServer)

Each **PaxosServer** simultaneously plays all three PMMC roles:

Logical Role	Responsibility
Replica	Receives client requests; proposes commands to fill log slots; applies committed entries to the local app state; responds to clients
Leader	Runs Phase 1 (ballot acquisition) and Phase 2 (value proposal) of Paxos; manages slot assignments
Acceptor	Votes on proposals; tracks the highest ballot it has promised and the highest value it has accepted per slot

Because all roles share the same process, the "acceptor" can short-circuit some messages internally without going over the network.

2.2 Data Structures

```

Ballot:      (round: int, leaderID: ServerID) -- lexicographically ordered
PValue:      (ballot: Ballot, slot: int, command: Command)
LogEntry:    { acceptedBallot, acceptedValue, decided: bool, decidedValue }
ServerState:
  myBallot    Ballot      -- highest ballot this node has seen / proposed
  isActive    bool        -- true if this server believes it is the leader
  log         Map<int, LogEntry>
  slot_in     int         -- next slot to propose (highest proposal + 1)
  slot_out    int         -- next slot to execute (oldest undecided)
  firstNonCleared int      -- lowest slot still in memory (GC cursor)
  decisions   Map<int, Command>
  proposals   Map<int, Command>
  clientReplies Map<(clientID, seqNo), Result> -- deduplication cache

```

2.3 Messages

Message	Sender → Receiver	Contents
<code>PaxosRequest</code>	Client → All Servers	<code>(clientID, seqNo, op)</code>
<code>PaxosReply</code>	Server → Client	<code>(seqNo, result)</code>
<code>P1A</code> (Prepare)	Active leader → All acceptors	<code>(ballot)</code>
<code>P1B</code> (Promise)	Acceptor → Leader	<code>(ballot, acceptorID, acceptedPValues[], decidedSlots[])</code>
<code>P2A</code> (Accept)	Active leader → All acceptors	<code>(ballot, slot, command)</code>
<code>P2B</code> (Accepted)	Acceptor → Leader	<code>(ballot, slot, acceptorID)</code>
<code>Decision</code>	Leader → All replicas	<code>(slot, command)</code>
<code>Heartbeat</code>	Active leader → All followers	<code>(leaderBallot, leaderSlotOut, gcSlot)</code>
<code>HeartbeatReply</code>	Follower → Leader	<code>(followerID, followerSlotOut)</code>

2.4 Timers

Timer	Owner	Period	Action
<code>HeartbeatSender</code>	Active leader	~50 ms	Broadcast <code>Heartbeat</code>
<code>HeartbeatChecker</code>	All servers	~100 ms	If 2 consecutive ticks with no heartbeat from known leader → start Phase 1
<code>ClientTimer</code>	Client	~200 ms	Rebroadcast unanswered <code>PaxosRequest</code> to all servers

2.5 Phase 1 — Leader Election (Scout)

Triggered when a server's `HeartbeatChecker` fires twice without hearing from the current leader, or when a node starts up.

1. Increment ballot: `myBallot = Ballot(myBallot.round + 1, myID)`.
2. Broadcast `P1A(myBallot)` to all other servers.
3. Each acceptor receiving `P1A(b)`:
 - If `b >= myBallot`: update `myBallot = b`, set `isActive = false`, reply with `P1B(b, self, acceptedPValues[slot >= firstNonCleared], decidedSlots)`.
 - Else: reply with `P1B(myBallot, self, ...)` — the preempt signal.
4. The prospective leader collects `P1B` messages. Upon receiving a **quorum (majority)**:
 - Check all returned ballots. If any `b' > myBallot`, set `isActive = false` (preempted) and return to follower mode.
 - Otherwise: set `isActive = true`. For each slot reported with an accepted pvalue, adopt the value from the highest ballot seen. Fill gaps (holes) with a no-op or pending client command.
 - Advance `slot_in` to `max(all reported slots) + 1`.

Invariant: After a successful Phase 1 with quorum Q, the leader knows all pvalues accepted by any member of Q. Any value previously decided (accepted by a prior quorum) is guaranteed to appear in at least one `P1B` in Q.

2.6 Phase 2 — Value Proposal (Commander)

For each slot `s` needing a decision (starting from `slot_out`):

1. Leader selects a command (either one recovered from Phase 1 or a new client request).
2. Broadcast `P2A(myBallot, s, command)` to all acceptors.
3. Each acceptor receiving `P2A(b, s, cmd)`:
 - If `b >= myBallot` and `s >= firstNonCleared`: record `(b, cmd)` in `log[s]`, reply `P2B(b, s, self)`.
 - Else: send `P2B(myBallot, s, self)` — preempt / stale rejection.
4. Leader collects `P2B` for slot `s`. Upon quorum of matching ballot:
 - Mark `log[s]` as decided. Broadcast `Decision(s, command)` to all replicas.
5. Upon receiving a `Decision(s, cmd)` (including self):
 - Set `log[s].decided = true, log[s].decidedValue = cmd`.
 - While `log[slot_out]` is decided: execute command, advance `slot_out`, send `PaxosReply` to client if deduplication cache shows this is the first execution.

2.7 Handling Concurrent Slots

The leader proposes to multiple slots concurrently rather than sequentially. It maintains a sliding window of in-flight proposals (`proposals` map). As long as the leader is not preempted, it

keeps filling slots with pending client commands and retrying any unacknowledged `P2A` messages on each `HeartbeatSendTimer` tick.

2.8 Stable Leader and Heartbeats

- The active leader broadcasts `Heartbeat` periodically. Followers reset their check timer on receipt.
- A `Heartbeat` also carries `gcSlot` (the global GC watermark) and `leaderSlotOut`, enabling GC and catch-up.
- When preempted (receives a ballot `b > myBallot`), the leader sets `isActive = false` and updates `myBallot = b`. It **does not** immediately restart Phase 1 — it waits for the heartbeat check to trigger if the new leader fails.

2.9 Garbage Collection

1. Each `HeartbeatReply` from a follower includes `followerSlotOut`.
2. The leader computes `gcSlot = min(all followerSlotOuts, leaderSlotOut)`. This is the highest slot executed on **all** servers.
3. `gcSlot` is broadcast in subsequent `Heartbeat` messages.
4. On receipt, each server sets `firstNonCleared = gcSlot + 1` and deletes `log[s]` for all `s <= gcSlot`.
5. **Catch-up:** If the leader sees that a follower's `followerSlotOut` lags, it sends missing `Decision` messages for slots in `[followerSlotOut, leaderSlotOut)`.

Note: GC requires all servers to communicate. It does not make progress if any server is unreachable, which is an accepted limitation of this simplified approach.

2.10 Quorum Requirements Summary

Phase	Required Quorum
Phase 1 (P1B collection)	Majority: $\text{floor}(n/2) + 1$
Phase 2 (P2B collection)	Majority: $\text{floor}(n/2) + 1$
GC watermark advancement	All <code>n</code> servers

2.11 Client Deduplication

Client requests carry (`clientID`, `seqNo`). Before executing, the server checks a `clientReplies` cache. If `seqNo` has already been executed, it returns the cached result without re-executing. This ensures **at-most-once** execution semantics at the application level, providing linearizability even if a client retransmits after a timeout.

3. Safety Claims and Liveness Assumptions

3.1 Safety: Agreement

Claim: No two servers ever decide different values for the same log slot.

Argument: This follows directly from classic single-decree Paxos:

- Phase 1 ensures that any value accepted by a previous quorum is learned by the new leader before Phase 2 begins (quorums overlap by at least one node).
- Phase 2 commits a value only after a quorum accepts it under the same ballot.
- Because quorums overlap, any two committed values for the same slot must have been proposed under the same or different ballots. If different ballots, the higher ballot's Phase 1 would have seen the previously accepted value and adopted it. Therefore both quorums decided the same value.

Claim: Only committed (decided) commands are executed, in slot order.

Argument: `slot_out` only advances when `log[slot_out].decided == true`.

Commands are applied sequentially from `slot_out`, so all servers apply the same sequence.

3.2 Safety: Validity

Every decided value was proposed by some leader, which received it from some client (or is a no-op used to fill a gap). No value is invented.

3.3 Safety: Linearizability

Given agreement (same log on all servers) and deduplication (at-most-once execution), the system is equivalent to a single sequential server processing commands in log order. This satisfies linearizability.

3.4 Liveness Assumptions

Liveness is **not guaranteed** unconditionally in asynchronous systems (FLP impossibility). We make the following assumptions for progress:

- **Partial synchrony:** Eventually, message delays are bounded enough that a leader can collect quorum responses before timing out and being preempted.
- **Majority connectivity:** At least $\text{floor}(n/2) + 1$ servers can communicate with each other and with at least one client.
- **Leader stability:** The heartbeat/check timer mechanism tends to elect a single stable leader. Dueling leaders slow progress but never violate safety.
- **No guarantee for GC:** Garbage collection requires all-server connectivity and is not required for liveness of command processing.

3.5 Leader Election Correctness

The ballot ordering (round , serverID) is a total order, ensuring that two concurrent Phase 1 attempts produce distinct ballots. Whichever leader's P1A messages arrive first at a quorum may succeed; the other will be preempted when it collects P1B responses containing the higher ballot.

4. Observability Plan

4.1 What to Instrument

The following state and events should be recorded (at DEBUG level in production; disable before submission per spec):

Leader Election Events

- Node transitions: $\text{FOLLOWER} \rightarrow \text{ACTIVE}$ (with new ballot) and $\text{ACTIVE} \rightarrow \text{FOLLOWER}$ (with preempting ballot)
- Number of consecutive heartbeat check timer fires without a heartbeat
- P1B responses collected per Phase 1 attempt: count, ballot seen, pvalues recovered

Phase 2 Progress

- Per slot: time from first P2A send to Decision
- Slots in the proposals map at each heartbeat (measures backlog)
- P2B rejections (ballot mismatch): indicates contention

Log State

- slot_out and slot_in on each server periodically — large gap suggests a "hole"
- firstNonCleared — confirms GC is advancing
- Set of undecided slots between slot_out and slot_in — holes blocking execution

GC Health

- Per follower: `followerSlotOut` received by leader — identifies lagging nodes
- `gcSlot` broadcast value over time — should increase monotonically

Client-facing

- Request round-trip latency from client perspective
- Number of `ClientTimer` retransmissions per request — high count indicates server-side stall
- Deduplication cache hits — should be rare in normal operation

4.2 Debugging Non-Termination

If commands stop being decided, check in order:

1. **Is there a leader?** Look for `HeartbeatSendTimer` firings. If none, all servers are in follower mode — check why Phase 1 is failing (not reaching quorum, or all P1B responses carry a higher ballot).
2. **Is Phase 1 completing?** Count P1B responses per Phase 1 attempt. If stuck at < majority, there is a network partition or bug in P1B sending.
3. **Are there log holes?** If `slot_out < slot_in` and `log[slot_out]` is not decided, there is a hole blocking execution. The leader should be proposing a value for that slot. Check that `P2A` messages are being sent for the hole slot.
4. **Is the leader being preempted immediately?** If every Phase 2 attempt produces P2B with a higher ballot, two servers are dueling. Verify heartbeat check timers are resetting on heartbeat receipt.
5. **Is GC blocking progress?** GC stalling does not block command processing — only `firstNonCleared` advancing requires all servers. Confirm `slot_out` is still advancing even if `gcSlot` is stuck.

4.3 Debugging Conflicting Decisions (Safety Violations)

A safety violation (two decided values for the same slot) should be impossible but would manifest as:

- Two servers returning different results to a client for the same `seqNo`
- `decidedValue` fields differing in `log[s]` across servers

To debug if suspected:

- Log the full `(slot, ballot, value)` triple at every `Decision` broadcast and receipt
- Log the complete `pvalues` adopted after every successful Phase 1

- Verify that no P2A is sent for a slot with a value that contradicts an already-adopted pvalue from Phase 1
- Check for off-by-one errors in `firstNonCleared` — accepting a P2A for a GC'd slot and then ignoring it can create a phantom gap

4.4 Key Metrics to Track in Testing

Metric	Normal Range	Alarm Condition
Leader elections per minute	< 2	> 10 (timer misconfiguration or partition)
P2A retransmissions per slot	0–1	> 5 (leader instability)
<code>slot_in - slot_out</code>	< 10	> 100 (execution blocked on hole)
<code>slot_out</code> delta per second	> 0 during load	0 for > 2× heartbeat period
Dedup cache hit rate	< 1%	> 10% (clients retransmitting excessively)
GC slot delta per second	> 0 when all nodes up	0 when all nodes reachable (bug)
